

Dealing with Imbalanced Text Data in Machine Learning



Salman Sigari · [Follow](#)

4 min read · Apr 16, 2023



Listen

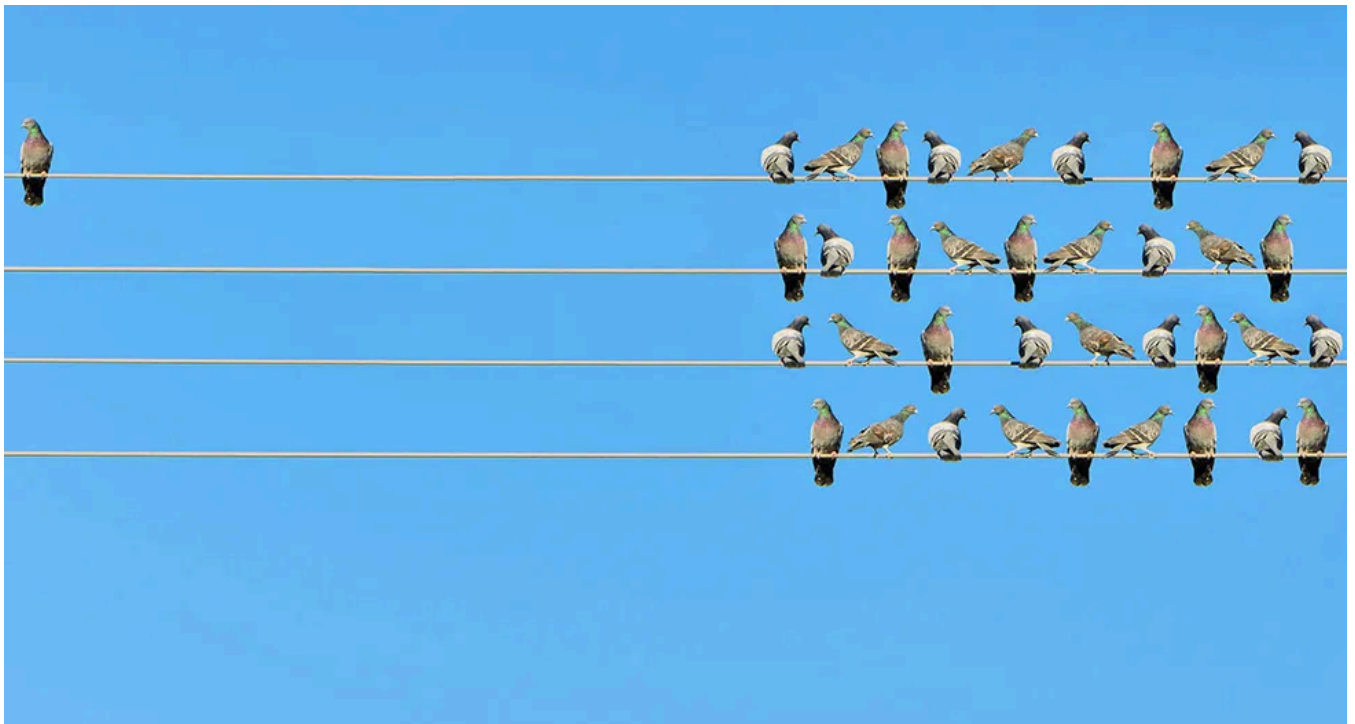


Share



More

Techniques and a comprehensive case study



Imbalanced text data is a common challenge in machine learning, particularly in natural language processing (NLP) tasks. It occurs when the distribution of classes in the target variable is not uniform, leading to poor model performance on minority classes. This article discusses various techniques for dealing with imbalanced text data, provides formulas and charts for better understanding, and offers a Python example to demonstrate the application of these techniques.

Techniques for Handling Imbalanced Text Data

1. Resampling Methods

a) Oversampling

This technique involves creating copies or synthetic instances of the minority class to balance the class distribution. Mathematically, let N_{maj} and N_{min} be the number of instances in the majority and minority classes, respectively. The oversampling process generates $(N_{maj} - N_{min})$ synthetic instances of the minority class.

b) Undersampling

This technique involves removing instances from the majority class to balance the class distribution. Mathematically, let N_{maj} and N_{min} be the number of instances in the majority and minority classes, respectively. The undersampling process randomly selects and removes $(N_{maj} - N_{min})$ instances from the majority class.

2. Cost-sensitive Learning

Cost-sensitive learning assigns different misclassification costs to each class, penalizing the model more for misclassifying minority class instances. Let C_{maj} and C_{min} be the misclassification costs for the majority and minority classes, respectively. The total cost function for a model is given by:

*Total_cost = $C_{maj} * (\text{number of majority class misclassifications}) + C_{min} * (\text{number of minority class misclassifications})$*

3. Ensemble Methods

a) Bagging: Bagging (Bootstrap Aggregating)

Open in app ↗



Search



balanced bagging, where each bootstrap sample has a balanced class distribution.

b) Boosting

Boosting involves training a sequence of weak learners, each focusing on the mistakes made by the previous learner. In the context of imbalanced data, cost-

sensitive boosting can be employed by assigning higher weights to misclassified minority instances. The final prediction is a weighted combination of the weak learners' predictions.

c) Random UnderSampling (RUS) Boost

RUS Boost combines boosting and undersampling. At each iteration, a random subset of majority class instances is selected to create a balanced dataset. A weak learner is then trained on this balanced dataset.

Metrics

1. **Confusion Matrix:** A confusion matrix is a table that shows the number of true positive (TP), true negative (TN), false positive (FP), and false negative (FN) predictions made by the model. It provides an overview of the model's performance, enabling us to understand the impact of applying different techniques to handle imbalanced data.
2. **Precision, Recall, and F1-score:** These metrics are particularly useful for evaluating the performance of models on imbalanced datasets.

$$\text{Precision} = TP / (TP + FP)$$

$$\text{Recall} = TP / (TP + FN)$$

$$\text{F1-score} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$$

Classification example

For this example, we'll use the 20 Newsgroups dataset, a popular dataset for text classification. We will create an imbalanced dataset by selecting two categories: 'rec.sport.baseball' (majority class) and 'sci.space' (minority class).

First, let's import necessary libraries and load the dataset:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix

categories = ['rec.sport.baseball', 'sci.space']
newsgroups = fetch_20newsgroups(subset='all', categories=categories)
```

Next, create the imbalanced dataset and preprocess the text data:

```
X = newsgroups.data
y = newsgroups.target

# Create an imbalanced dataset
imbalanced_indices = np.where(y == 0)[0].tolist() + np.where(y == 1)[0][:100].tolist()
X_imbalanced = [X[i] for i in imbalanced_indices]
y_imbalanced = y[imbalanced_indices]

# Preprocess text data
vectorizer = TfidfVectorizer()
X_imbalanced_vectorized = vectorizer.fit_transform(X_imbalanced)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X_imbalanced_vectorized, y_imbalanced,
```

Now, let's apply the SMOTE (Synthetic Minority Over-sampling Technique) for oversampling and train a classifier

```
from imblearn.over_sampling import SMOTE
from sklearn.naive_bayes import MultinomialNB

smote = SMOTE()
X_train_resampled, y_train_resampled = smote.fit_resample(X_train, y_train)

classifier = MultinomialNB()
classifier.fit(X_train_resampled, y_train_resampled)

y_pred = classifier.predict(X_test)

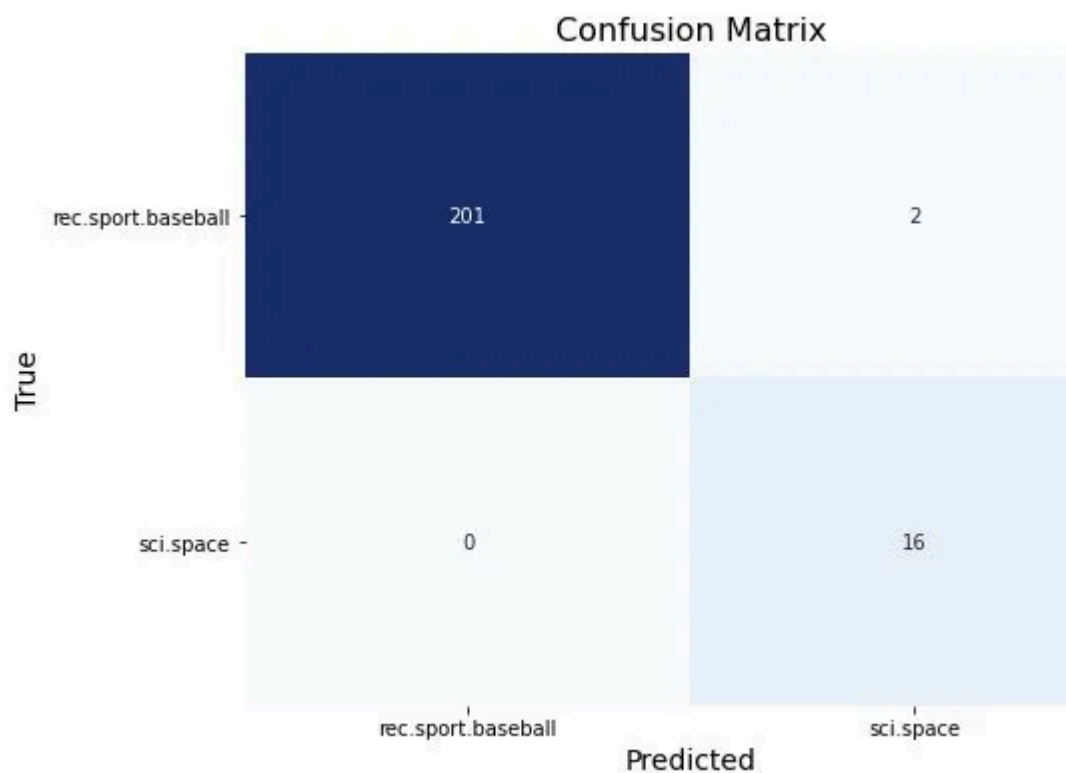
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	1.00	0.99	1.00	203
1	0.89	1.00	0.94	16
accuracy			0.99	219
macro avg	0.94	1.00	0.97	219
weighted avg	0.99	0.99	0.99	219

To visualize the confusion matrix, use the following code:

```
def plot_confusion_matrix(y_true, y_pred):
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(8, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)
    plt.xlabel('Predicted', fontsize=14)
    plt.ylabel('True', fontsize=14)
    plt.xticks([0.5, 1.5], ['rec.sport.baseball', 'sci.space'])
    plt.yticks([0.5, 1.5], ['rec.sport.baseball', 'sci.space'], rotation=0)
    plt.title('Confusion Matrix', fontsize=16)
    plt.show()

plot_confusion_matrix(y_test, y_pred)
```



In this example, we demonstrated how to handle imbalanced text data using the SMOTE technique and visualized the model's performance using a confusion

matrix. Other techniques mentioned in this article can also be applied depending on the specific problem and dataset.

Conclusion

Handling imbalanced text data is crucial for improving the performance of machine learning models in NLP tasks. Techniques such as resampling, cost-sensitive learning, and ensemble methods can be employed to mitigate the effects of class imbalance. By implementing these techniques, practitioners can ensure better model performance on minority classes, leading to more accurate and reliable predictions.

Thanks for Reading!

Follow me to receive my latest posts if you enjoyed this post!

You can find the code here

GitHub - salman6049/Dealing-with-Imbalanced-Text-Data-in-Machine-Learning

Imbalanced text data is a common challenge in machine learning, particularly in natural language processing (NLP)...

github.com

Machine Learning

Data Science

Generative Ai

Quality

Artificial Intelligence



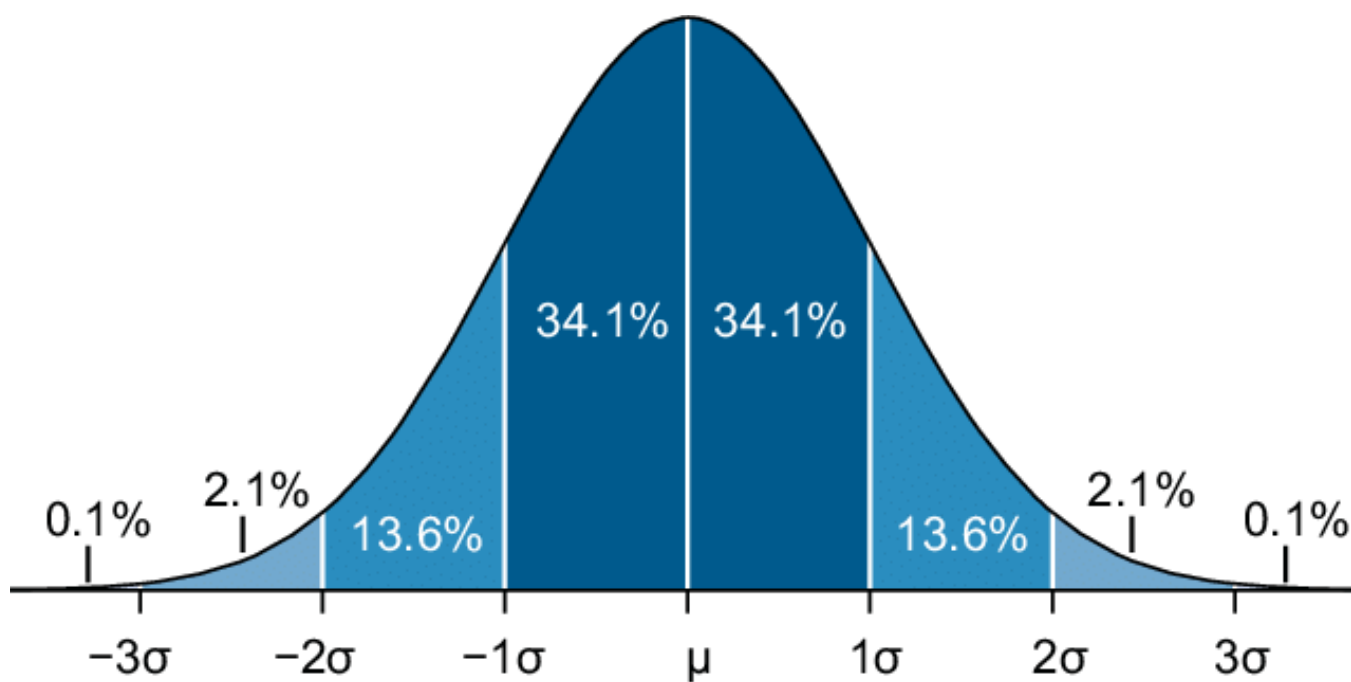
Follow

Written by Salman Sigari

32 Followers

Data Scientist - All opinions are mine.

More from Salman Sigari



Salman Sigari

Discovering the Best-fit Probability Distribution for Your Data

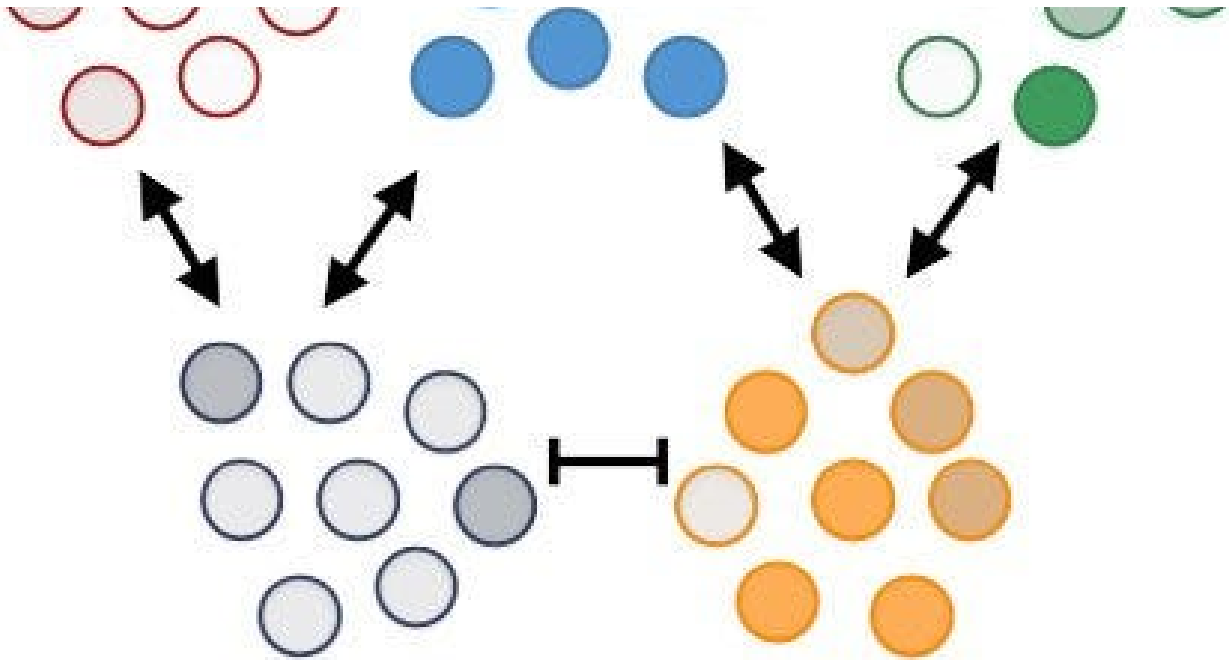
An Analysis using Python

4 min read · Apr 9, 2023



52





 Salman Sigari

Probabilistic Learning: A Deep Dive with Python

Within the vast realm of machine learning, probabilistic learning has carved out its own unique space. Driven by statistics and...

7 min read · Jun 2, 2023

 12 



 Salman Sigari

Striking the Balance Between Data Quality and Quantity in Machine Learning

In the era of big data, machine learning (ML) has emerged as a powerful tool for extracting insights and making predictions. As ML...

4 min read · Mar 16, 2023



40



Salman Sigari

Deciphering the Art of AI: How Large Language Models Mimic Human Translation

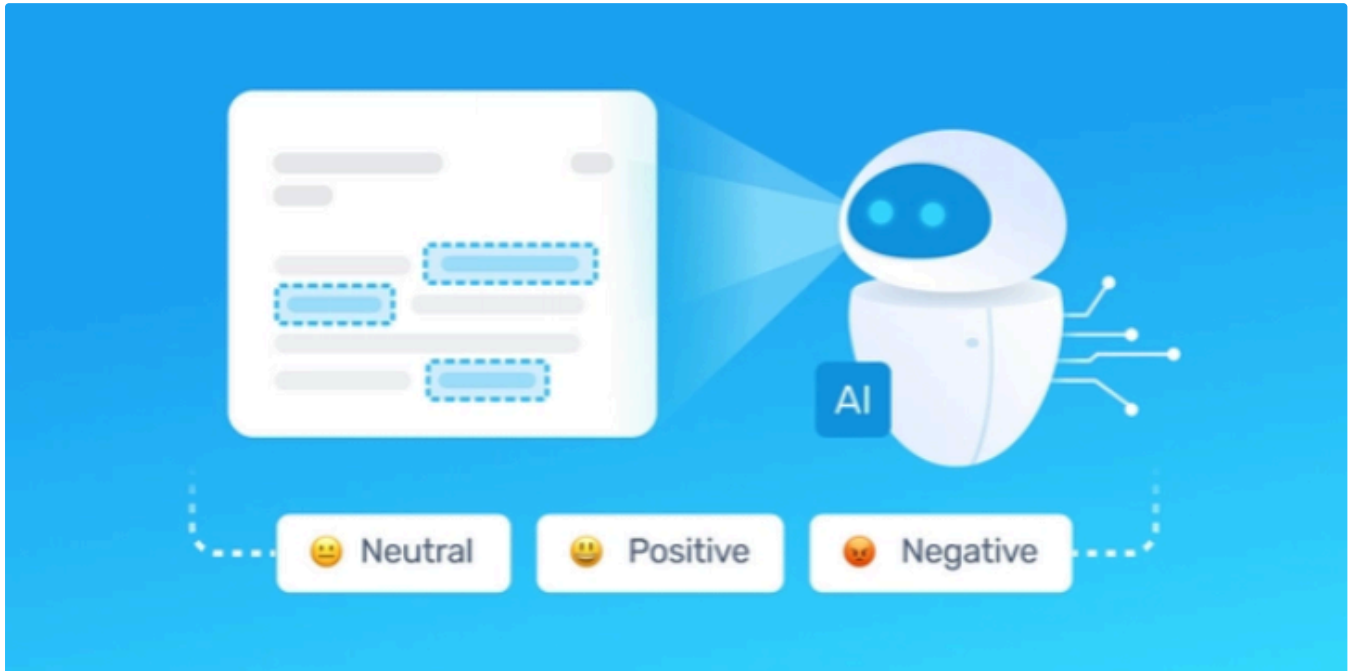
The digital landscape of language translation has rapidly evolved, with platforms like Google Translate enabling global conversations and...

12 min read · May 27, 2023



See all from Salman Sigari

Recommended from Medium



 Prakash Ramu

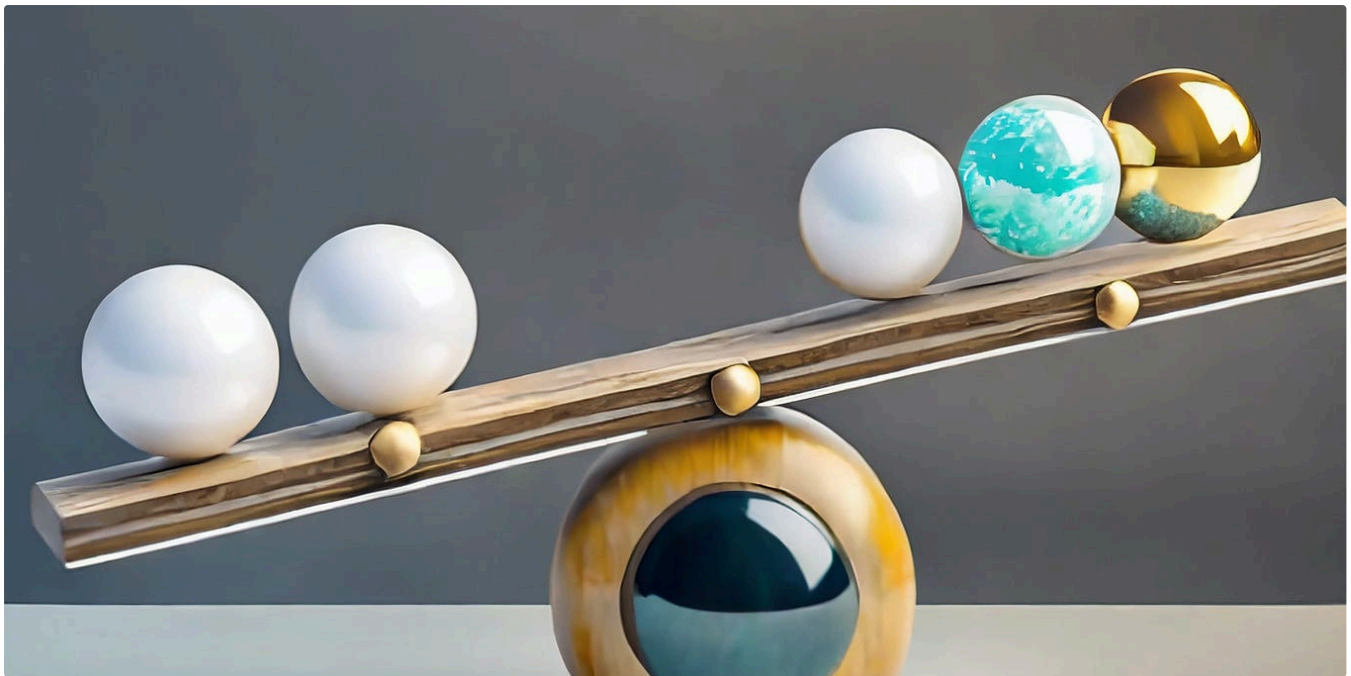
Building a Text Classification Model using DistilBERT

In this blog post, we'll walk through the process of building a text classification model using the DistilBERT model. Text classification...

4 min read · Mar 29, 2024

 60  1



Manoj Kumar Vohra

Sequence classification techniques for imbalanced datasets (Part-1)

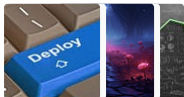
6 min read · Dec 30, 2023



62



Lists



Predictive Modeling w/ Python

20 stories · 1309 saves



Natural Language Processing

1531 stories · 1065 saves



Practical Guides to Machine Learning

10 stories · 1577 saves



AI Regulation

6 stories · 485 saves



 Sagar Dubey

Sentence Transformers can be your answer to resource hungry LLMs

There are both cost and latency constraints around using LLMs. What if you can get comparable performance by using pint-sized models?

9 min read · Dec 28, 2023

 4 



 Fatmanurkutlu

Model Evaluation Techniques in Machine Learning

What is Model Evaluation?

5 min read · Jan 11, 2024



 Abhishek Jain

Label Encoding and One- Hot Encoding

In this blog, we are gonna look at Label Encoding and One-Hot Encoding which are used to convert categorical data into numerical data

2 min read · Jan 7, 2024





Sabaybiometzger

Fine-Tuning GEMMA-2b for Binary Classification (4-bit Quantization)

Introduction

10 min read · May 8, 2024



3



1



See more recommendations